THOMAS MORE

IQ Noodles Puzzle Assistant

Reflection

Abdul Salam Aldabik
Student Bachelor Applied Computer Science

2025-2026

Table of contents

1. Introduction

This document is the reflection on my internship at Smart NV (23 February to 22 May 2026), where I built the IQ Noodles puzzle assistant. The technical result is described in the separate realization document; here I step back from the code to look at the work itself. The first part is a substantive reflection on what the project delivered, what it is worth to the client and its users, and what is left to do. The second part is a personal reflection on what the internship meant to me, what I learned, and the problems I ran into and how I dealt with them.

2. Substantive reflection

The internship delivered a working, on-device computer-vision system for IQ Noodles, starting from nothing: there was no dataset, no trained model and no application at the outset. By the end there was a synthetic-data generator in Blender, a YOLO segmentation model trained in two phases, a full vision pipeline that turns a phone photo into a board state in the browser, a backtracking solver with hints, and a mobile-first app that ties it together. The model reaches a mask mAP@0.5 of about 0.94 on real photos, and the solver clears a board in tens of milliseconds.

For Smart NV, the value is twofold. The obvious part is a prototype that shows their idea is feasible: a phone can read a physical board and help a child who is stuck, without sending any photo to a server, which keeps the whole thing inside GDPR for images of children. The less obvious part is everything created along the way that the company did not have before: a labelled dataset, a repeatable way to generate more synthetic data, and a clear picture of which pieces are hard to detect and why. For the users, a child and the parent next to them, the benefit is a hint that nudges rather than a booklet that spoils the answer.

The project is at prototype stage and is not fully finished. The core is solid, but a few things remain. The model is weakest on the pieces whose colour sits close to the dark board, so more real photos of those pieces under harder lighting would help. The browser-only parts of the app are tested by hand rather than by an automated harness. The system has been validated on more than sixty real scan sessions but has not yet been put in front of real users in the field. My advice to the client for the next phase is to invest first in a larger and more varied real dataset, since almost every remaining accuracy gap traces back to data rather than to the model or the code, and then to run a small user test with children in the target age group before thinking about a production release.

3. Personal reflection

This internship was the first time I took a machine-learning idea all the way from an empty folder to something that runs on a phone, and that end-to-end ownership is what made it valuable to me. I had to be the data engineer, the model trainer, the computer-vision developer and the front-end developer at different moments, and switching between those roles taught me more than any single one of them would have on its own.

On the technical side I learned a lot. I trained and fine-tuned a YOLO segmentation model and came to understand the gap between synthetic and real data the hard way, when an early synthetic-only model detected only three pieces out of five. I wrote a full classical vision pipeline by hand, including a least-squares homography and a colour classifier in CIE L*a*b* space, because the browser has no OpenCV. I learned that in a constraint solver the intelligence lives in the variable ordering, not the search loop: the right heuristic turned a twenty-four-second solve into a thirty-millisecond one. I also got better at working with AI coding tools, mostly by learning to feed them the right context and to cross-check a diagnosis against more than one model.

Not all of the problems were technical. The hardest stretch was a run of days where the scan pipeline kept mapping pieces to the wrong cells and no amount of tuning fixed it. I kept patching a broken foundation instead of stepping back, and the lesson I took from it, after finally deciding to redesign the geometry from a single source of truth, was to recognise when an approach has a ceiling and to stop digging. I had a similar moment with the synthetic-data tool: PyRender simply could not produce what I needed, and moving to Blender was the right call even though it meant throwing work away. One of the most useful unblocks of the whole project, raising detections from one or two per scan to ten or more, came not from the geometry I was blaming but from a decoder reading the model's output in the wrong format, which taught me to check a component in isolation before assuming the fault is downstream.

I also learned something about managing pressure and expectations. When the workload peaked and the project felt overwhelming, I had an honest conversation with my mentor and we set a clear checkpoint for it to either show progress or be re-scoped. Saying out loud that I was stuck, rather than quietly falling behind, turned out to be the more professional move, and the project came through that checkpoint. Alongside that I learned smaller but real lessons about infrastructure, like moving training off a cloud notebook that kept timing out and onto a self-hosted machine with sessions that survive a dropped connection.

Looking back, the internship grew me from someone who could use machine-learning tools into someone who can carry a vision project end to end and make engineering judgements under real constraints: choosing robustness over elegance, building debugging and calibration tools early, and backing out of a wrong decision quickly instead of defending it. I am proud of what the system does, clear-eyed about what it still needs, and more confident than before that I can pick up an unfamiliar, messy problem and turn it into something that works.